

Python Basics Exam Preparation Guide

(Assignments 25-39)

This guide covers all key topics for your exam tomorrow, mapped directly to your assignments from Day 25 onwards. Study the logic, templates, and patterns below.

Topic Breakdown & Quick Navigation

Topic	Assignment Days	Key Concepts	Files Link
1. Strings	25, 26, 27, 28, 29, 30	Validation, Parsing, Compression, Reversals, Cleaning	25_18May/assi1.py
2. Lists & Nested Lists	31, 32, 33, 34, 36	Peak Finding, Leaders, Comprehension, Rotations	31_28May/assi1.py
3. Tuples & NamedTuples	37	Immutability, namedtuple grouping, Aggregation	37_5Jun/assi1.py
4. Sets & Frozensets	38	Venn Operations, Immutability, Nesting using frozenset	38_8Jun/assi1.py
5. Dictionaries & Nested Dicts	39	Frequency Counting, Grouping, Nested Lookups	39_11Jun/assi1.py
6. 3D Matrices	34, 35	Nested Loops, 3D Indexing, Traversal, Reductions	34_1Jun/assi4.py



Topic 1: Strings (Days 25-30)

Core Concepts

- **Immutability:** Strings cannot be modified in-place. All transformations return new strings.
- **Basic Methods:** `split()`, `join()`, `isdigit()`, `isalpha()`, `isalnum()`, `startswith()`, `endswith()`.
- **Custom Implementations:** Often, tests forbid using `title()`, `reverse()`, or `Counter`. You must write manual logic.



Key Patterns to Remember

- **Manual Title Case (No `.title()`):**

```
words = s.split()

formatted = [w[0].upper() + w[1:] for w in words]

result = " ".join(formatted)
```

- **Masking Strings (e.g., Bank Cards):**

```
masked = "*" * (len(card) - 4) + card[-4:]
```

- **Run-length Expansion (`a3b2` -> `aaabbb`):**

```
expanded = ""

for i in range(0, len(s), 2):

    char = s[i]

    count = int(s[i+1])
```

```
expanded += char * count
```



Topic 2: Lists & Nested Lists (Days 31-36)

Core Concepts

- **Peak Element:** An element which is greater than or equal to its neighbors.
 - For edge elements, compare with the single neighbor:

```
if i == 0: is_peak = arr[0] >= arr[1]

elif i == n - 1: is_peak = arr[-1] >= arr[-2]

else: is_peak = arr[i] >= arr[i-1] and arr[i] >= arr[i+1]
```

- **Leader in Array:** An element greater than all elements to its right.
 - Traverse from right to left to solve in $O(N)$ time:

```
max_so_far = float('-inf')

leaders = []

for num in reversed(arr):

    if num > max_so_far:

        leaders.append(num)

        max_so_far = num

leaders.reverse()
```

- **List Comprehensions:**
 - Syntax: `[expr for item in list if condition]`

- Bonus Application: `[sal + 5000 if sal < 30000 else sal for sal in salaries]`



Topic 3: Tuples & NamedTuples (Day 37)

Core Concepts

- **Tuples:** Ordered, immutable collections. Defined by `(item1, item2, ...)`. Used when records should not change.
- **NamedTuple:** Part of `collections` module. It combines tuple immutability with named attribute access.

```
from collections import namedtuple
```

```
Employee = namedtuple('Employee', ['emp_id', 'emp_name', 'department', 'salary'])
```

☀ Key Patterns

- **Reading List of NamedTuples:**

```
employees = []
```

```
for _ in range(n):
```

```
    eid, name, dept, sal = input().split()
```

```
    employees.append(Employee(int(eid), name, dept, float(sal)))
```

- **Finding Max by attribute:**

```
highest_sal_emp = max(employees, key=lambda emp: emp.salary)
```

Topic 4: Sets & Frozensets (Day 38)

Core Concepts

- **Set:** Unordered collection of unique items. Mutable, but items must be hashable.
- **Frozenset:** Immutable version of set. Can be used as a dictionary key or placed inside another set.
- **Set Operations:**
 - **Union** (`|` / `union()`): All unique members across sets.
 - **Intersection** (`&` / `intersection()`): Members in both sets.
 - **Difference** (`-` / `difference()`): Members in first set but not the second.
 - **Symmetric Difference** (`^`): Members in either set, but not both.

How to Nest Sets

Since sets are mutable, you *cannot* have a set of sets (`{{1, 2}, {3, 4}}` raises `TypeError`). You must nest using `frozenset`:

```
nested_set = {frozenset({1, 2}), frozenset({3, 4})}
```



Topic 5: Dictionaries & Nested Dicts (Day 39)

Core Concepts

- **Dict:** Key-value pairs. Keys must be unique and hashable.
- **Nested Dictionary:** Values themselves are dictionaries. Great for structured databases.

```
db = {  
    "emp1": {"name": "Ajay", "dept": "IT", "salary": 50000},  
    "emp2": {"name": "Priya", "dept": "HR", "salary": 45000}  
}
```



Key Patterns

- **Frequency Counting without Counter:**

```
freq = {}  
  
for item in items:  
    freq[item] = freq.get(item, 0) + 1
```

- **Grouping by length (Dict of Lists):**

```
grouped = {}  
  
for word in words:
```

```
l = len(word)
```

```
grouped.setdefault(l, []).append(word)
```



Topic 6: 3D Matrices (Days 34-35)

Core Concepts

- A 3D Matrix is a list of lists of lists. It represents dimensions: **Depth x Rows x Columns** (or **Blocks x Rows x Cols**).
- **Initialization** (e.g., $2 \times 3 \times 4$ filled with 0s):

```
matrix_3d = [[[0 for _ in range(cols)] for _ in range(rows)] for _ in range(depth)]
```

- **Traversing a 3D Matrix:**

```
for d in range(depth):
```

```
    for r in range(rows):
```

```
        for c in range(cols):
```

```
            print(matrix_3d[d][r][c])
```



Aggregate Computations

- **Sum of all elements:**

```
total_sum = sum(matrix_3d[d][r][c] for d in range(depth) for r in range(rows) for c in range(cols))
```

- **Plane-wise (Depth-wise) Sum:**

```
plane_sums = [sum(sum(row) for row in plane) for plane in matrix_3d]
```

Solved Exam Practice Questions (One-by-One Review)

Section 1: Strings

 **IMPORTANT: Question 1:** Read an alphanumeric string from the user. Clean the string by removing all special characters (keep only letters, digits, and spaces). Then, reverse each word individually, but keep the word positions unchanged. First letter of each reversed word should be capitalized.

Input: Pyth@n!! is ve#ry ea\$sy

Output: Nohtyp Si Yrev Ysae

Implementation Strategy

1. Remove all characters that are not `.isalnum()` or space.
2. Split the cleaned string into words.
3. For each word, reverse it (`word[::-1]`), lowercase the rest, and capitalize the first character (`.capitalize()`).
4. Join the words back with spaces.

```
def process_string(s):
```



```
# Step 1: Clean special characters
```

```
cleaned_chars = [char for char in s if char.isalnum() or char.isspace()]
```

```
cleaned_str = "".join(cleaned_chars)
```

```
# Step 2: Split and transform
```

```
words = cleaned_str.split()
```

```
transformed_words = []
```

```
for word in words:
```

```
    reversed_word = word[::-1]
```

```
    # Capitalize only first letter, keep others lowercase
```

```
    transformed_words.append(reversed_word.capitalize())
```

```
return " ".join(transformed_words)
```

```
# Example execution
```

```
user_input = "Pyth@n!! is ve#ry ea$sy"
```

```
print(process_string(user_input)) # Output: Nohtyp Si Yrev Ysae
```

Section 2: Lists & Nested Lists

 **IMPORTANT: Question 2:** You are given an array of elevation points of a hiker. Find all “peak” checkpoints (which are greater than or equal to their neighbors). Also, write a list comprehension to filter out hikers with speed below average and award a speed bonus of 10% to the remaining hikers.

Input: elevations = [100, 200, 150, 180, 170], speeds = [12, 15, 9, 20, 14]

Output: Peaks = [200, 180], Updated Speeds = [16.5, 22.0, 15.4] (since average speed is 14)

Implementation Strategy

1. Loop through list, checking boundaries.
2. For list comprehension, calculate average first, then apply conditional operation.

```
def find_peaks_and_boost(elevations, speeds):
```

```
    n = len(elevations)
```

```
    peaks = []
```

```
    # Peak detection logic
```

```
    for i in range(n):
```

```
        if n == 1:
```

```
            peaks.append(elevations[0])
```

```
        elif i == 0:
```

```
            if elevations[0] >= elevations[1]:
```

```
                peaks.append(elevations[0])
```

```
        elif i == n - 1:
```

```
        if elevations[n-1] >= elevations[n-2]:

            peaks.append(elevations[n-1])

        else:

            if elevations[i] >= elevations[i-1] and elevations[i] >= elevations[i+1]:

                peaks.append(elevations[i])

            # List comprehension speed boost

            avg_speed = sum(speeds) / len(speeds)

            # Give 10% boost to those with speed >= average speed

            boosted_speeds = [s * 1.1 for s in speeds if s >= avg_speed]

        return peaks, boosted_speeds

elevations = [100, 200, 150, 180, 170]

speeds = [12, 15, 9, 20, 14]

p, s = find_peaks_and_boost(elevations, speeds)

print("Peaks:", p)          # Peaks: [200, 180]

print("Boosted Speeds:", [round(val, 2) for val in s]) # Boosted Speeds: [16.5, 22.0, 15.4]
```

Section 3: Tuples & NamedTuples

 **IMPORTANT: Question 3:** Define a `Student` NamedTuple with fields `roll_no`, `name`, `course`, and `marks`. Read details of N students, store them, find the topper, and display the average marks of the class.

Input:

3

1 Ajay Python 85

2 Priya Java 90

3 Neha Python 95

Output:

Topper: 3 Neha Python 95

Average Marks: 90.0

```
from collections import namedtuple

# Define NamedTuple

Student = namedtuple('Student', ['roll_no', 'name', 'course', 'marks'])

def run_student_tracker():

    students = []

    # Hardcoded input simulation

    inputs = [

        "1 Ajay Python 85",

        "2 Priya Java 90",

        "3 Neha Python 95"
```

```

    ]

    for record in inputs:

        roll, name, course, marks = record.split()

        students.append(Student(int(roll), name, course, float(marks)))

    # Find topper

    topper = max(students, key=lambda s: s.marks)

    # Calculate average

    avg_marks = sum(s.marks for s in students) / len(students)

    print(f"Topper: {topper.roll_no} {topper.name} {topper.course} {topper.marks}")

    print(f"Average Marks: {avg_marks}")

run_student_tracker()

```

Section 4: Sets & Frozensets (with Nesting)

 **IMPORTANT: Question 4:** A college maintains sets of student IDs in `Coding_Club` and `Robotics_Club`. Find students in both clubs, students unique to coding, and total unique students across both. Also, create a nested set of frozen sets representing groups of students.

Input:

coding = {101, 102, 103, 104}

robotics = {103, 104, 105, 106}

Output:

Both: {103, 104}

Only Coding: {101, 102}

Total Unique: {101, 102, 103, 104, 105, 106}

```
def set_operations_demo():
```

```
    coding = {101, 102, 103, 104}
```

```
    robotics = {103, 104, 105, 106}
```

```
    # 1. Intersection (Both clubs)
```

```
    both = coding & robotics # or coding.intersection(robotics)
```

```
    # 2. Difference (Only Coding)
```

```
    only_coding = coding - robotics # or coding.difference(robotics)
```

```
    # 3. Union (All Unique Members)
```

```
    all_unique = coding | robotics # or coding.union(robotics)
```

```
    # 4. Nested sets (Must use frozenset for nesting!)
```

```
    # We want a set containing the groups
```

```
    group1 = frozenset(coding)
```

```

group2 = frozenset(robotics)

nested_set = {group1, group2}

print("Both Clubs:", both)

print("Only Coding:", only_coding)

print("All Unique Members:", all_unique)

print("Nested Set of Frozensets:", nested_set)

set_operations_demo()

```

Section 5: Dictionaries & Nested Dictionaries

 **IMPORTANT: Question 5:** You are given a list of employees in a company with their departments and salaries. Construct a nested dictionary where the outer key is the **department** and the value is a nested dictionary of **employee_name: salary**. Compute the highest paid employee in a requested department.

Input list of records: `[("IT", "Ajay", 60000), ("IT", "Ravi", 75000), ("HR", "Priya", 50000), ("HR", "Neha", 55000)]`

Query Department: `"IT"`

Output:

Nested Dictionary: `{'IT': {'Ajay': 60000, 'Ravi': 75000}, 'HR': {'Priya': 50000, 'Neha': 55000}}`

Highest Paid in IT: `"Ravi with 75000"`

```

def analyze_nested_dict(records, query_dept):

```

```
nested_db = {}
```

```
for dept, name, salary in records:
```

```
    if dept not in nested_db:
```

```
        nested_db[dept] = {}
```

```
        nested_db[dept][name] = salary
```

```
print("Nested Dictionary Structure:")
```

```
print(nested_db)
```

```
# Query logic
```

```
if query_dept in nested_db:
```

```
    dept_employees = nested_db[query_dept]
```

```
    # Find highest paid employee
```

```
    highest_paid = max(dept_employees, key=dept_employees.get)
```

```
    highest_salary = dept_employees[highest_paid]
```

```
    print(f"\nHighest Paid in {query_dept}: {highest_paid} with {highest_salary}")
```

```
else:
```

```
    print("Department not found")
```

```
records = [
```

```
    ("IT", "Ajay", 60000),
```



```
("IT", "Ravi", 75000),  
  
("HR", "Priya", 50000),  
  
("HR", "Neha", 55000)  
  
]  
  
analyze_nested_dict(records, "IT")
```

Section 6: 3D Matrices

 **IMPORTANT: Question 6:** Initialize a 3D Matrix of size $D \times R \times C$ (Depth, Rows, Columns) with user inputs. Perform the following operations: 1. Print the 3D Matrix plane-by-plane. 2. Find the global maximum element and its 3D coordinate index (**depth, row, col**). 3. Calculate plane-wise sums.

Input dimensions: $D = 2, R = 2, C = 3$

Elements:

Plane 0:

1 2 3

4 5 6

Plane 1:

7 8 9

10 11 12

```
def run_3d_matrix_analysis():  
  
    # Size dimensions  
  
    depth, rows, cols = 2, 2, 3  
  
  
  
    # Input simulation: Flat list of values to fill in
```

```
input_values = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
val_idx = 0
```

```
# 1. Initialization of 3D matrix using nested list comprehension
```

```
matrix_3d = [[[0 for _ in range(cols)] for _ in range(rows)] for _ in range(depth)]
```

```
# Fill the matrix
```

```
for d in range(depth):
```

```
    for r in range(rows):
```

```
        for c in range(cols):
```

```
            matrix_3d[d][r][c] = input_values[val_idx]
```

```
            val_idx += 1
```

```
# Display plane-by-plane
```

```
print("--- 3D Matrix Print ---")
```

```
for d in range(depth):
```

```
    print(f"Plane (Depth) {d}:")
```

```
        for r in range(rows):
```

```
            print(" ", matrix_3d[d][r])
```

```
# 2. Find global max with coordinates
```

```
max_val = float('-inf')
```

```
max_coord = (-1, -1, -1)
```

```
for d in range(depth):
```

```
    for r in range(rows):
```

```
        for c in range(cols):
```

```
            curr = matrix_3d[d][r][c]
```

```
            if curr > max_val:
```

```
                max_val = curr
```

```
                max_coord = (d, r, c)
```

```
print(f"\nGlobal Max Value: {max_val} at Coordinate (depth={max_coord[0]}, row={max_coord[1]}, col={max_coord[2]})")
```

```
# 3. Calculate plane-wise sums
```

```
plane_sums = []
```

```
for d in range(depth):
```

```
    plane_sum = 0
```

```
    for r in range(rows):
```

```
        plane_sum += sum(matrix_3d[d][r])
```

```
    plane_sums.append(plane_sum)
```

```
for d, psum in enumerate(plane_sums):
```

```
    print(f"Sum of Plane {d}: {psum}")
```

```
run_3d_matrix_analysis()
```



Top 5 Tips for the Test

1. **Never use prohibited built-ins:** If the question says “Do not use `title()`”, write the capitalize splits manually. If it says “Do not use `Counter`”, use the `.get(key, 0)` dictionary pattern.
2. **Handle array edge cases:** When analyzing neighbors (like peak check or leader check), always check bounds first so your code doesn’t crash with `IndexError`.
3. **Use Frozenset for nesting sets:** Normal sets cannot contain sets. Use `frozenset` when you need to group items inside an outer set.
4. **Be comfortable with list comprehensions:** Expect questions asking you to write a single-line list comprehension containing conditionals (e.g. `[x for x in list if condition]`).
5. **Remember 3D coordinates:** For 3D matrices, index order is always `[depth][row][col]`. When writing triple loops, keep this order to iterate correctly.

Good luck! 🚀